

MORRISON_HTTP/3+QUIC_DOC

Documentación Técnica del Módulo de Reverse Proxy (HTTP/1.1, HTTP/2 y HTTP/3 + QUIC)

Proyecto: Librería Pequeño Morrison — Plataforma de Comercio de Libros

ARQUITECTURA DEL MÓDULO

El módulo de Reverse Proxy actúa como único punto de entrada de toda la plataforma de la librería Pequeño Morrison. Ningún otro servicio del sistema (API REST, servidor de WebSockets, gRPC, SMTP, SFTP o TCP) es accesible de forma directa desde el navegador; toda solicitud proveniente del cliente web pasa primero por este componente, el cual la redirige internamente al servicio correspondiente.

El navegador establece una única conexión visible hacia el Reverse Proxy (Caddy). Este servidor negocia automáticamente con el navegador la mejor versión de HTTP disponible (HTTP/1.1, HTTP/2 o HTTP/3 sobre QUIC/UDP) y, una vez recibida la solicitud, la reenvía hacia el backend correspondiente utilizando el protocolo que dicho backend sí soporta (normalmente HTTP/1.1). El Reverse Proxy espera la respuesta del backend y la devuelve al navegador respetando la versión de HTTP originalmente negociada con el cliente.

El Reverse Proxy no implementa lógica de negocio, autenticación, acceso a bases de datos ni procesamiento de solicitudes. Su única responsabilidad es recibir conexiones desde el navegador y reenviarlas al servicio adecuado, actuando como intermediario de protocolo entre el cliente moderno (HTTP/3 capaz) y los servidores internos (limitados a HTTP/1.1).

COMPONENTES

- Servidor Reverse Proxy (Caddy): expone los puertos 80 (HTTP, redirección a HTTPS) y 443 (HTTPS/TCP para HTTP/1.1 y HTTP/2, y HTTPS/UDP para HTTP/3 vía QUIC).
- Gestor de Protocolo (ALPN / Alt-Svc): negocia con el navegador la versión de HTTP a utilizar y anuncia la disponibilidad de HTTP/3 mediante la cabecera Alt-Svc.
- Enrutador de Solicitudes: distribuye el tráfico entrante según la ruta solicitada, dirigiendo peticiones HTTP hacia la API REST y conexiones de actualización (upgrade) hacia el servidor de WebSockets.
- Backend — API REST: servicio interno que expone la lógica de negocio de la librería (registro, catálogo, compras). Sólo habla HTTP/1.1 y no es accesible directamente desde el navegador.
- Backend — Servidor de WebSockets: servicio interno de notificaciones en tiempo real (puerto 8765). Tampoco es accesible directamente; el proxy gestiona el handshake de actualización de protocolo (Upgrade: websocket).

DOCUMENTACIÓN DE LA CONEXIÓN DEL NAVEGADOR

URL Base

```
https://<host>
```

Métodos de Transporte Soportados

HTTP/1.1, HTTP/2 y HTTP/3 (QUIC sobre UDP). El navegador selecciona automáticamente la mejor versión disponible; no requiere configuración manual por parte del cliente.

Objetivo

Permitir que el navegador se comunice con la plataforma utilizando el protocolo de transporte más eficiente disponible, sin que esto afecte el comportamiento de la API REST ni del servidor de WebSockets, los cuales permanecen ajenos a la versión de HTTP utilizada por el cliente.

Requiere autenticación

No. La negociación de protocolo y el enrutamiento ocurren antes de cualquier verificación de identidad, la cual es responsabilidad exclusiva de la API REST.

Requiere autorización

No.

Parámetros de Negociación de Protocolo

Parámetro	Capa	Descripción
ALPN	TLS (puerto 443/TCP)	Extensión TLS utilizada para negociar HTTP/1.1 o HTTP/2 durante el handshake.
Alt-Svc	Cabecera HTTP de respuesta	Anuncia al navegador que el servidor también soporta HTTP/3 en el mismo host y puerto, permitiendo que futuras solicitudes usen QUIC.
QUIC	Transporte (puerto 443/UDP)	Protocolo de transporte sobre UDP utilizado por HTTP/3, con cifrado integrado y establecimiento de conexión más rápido.

Ejemplo de negociación

1. El navegador realiza la primera solicitud por HTTPS (TCP/443) usando HTTP/1.1 o HTTP/2.
2. Caddy responde incluyendo la cabecera Alt-Svc, anunciando soporte para HTTP/3 en el mismo puerto vía UDP.
3. En solicitudes posteriores, el navegador abre una conexión QUIC (UDP/443) y utiliza HTTP/3 automáticamente, sin intervención del usuario.

DOCUMENTACIÓN DEL ENRUTAMIENTO INTERNO

Base Path

/

Objetivo

Redirigir cada solicitud entrante hacia el servicio interno correspondiente, preservando la semántica HTTP (método, encabezados y cuerpo) independientemente de la versión de protocolo utilizada por el navegador.

Requiere autenticación

No. La autenticación es delegada y validada por la API REST.

Requiere autorización

No.

Tabla de Rutas

Ruta	Destino (backend)	Protocolo hacia backend	Descripción
/api/*	API REST (host interno:puerto)	HTTP/1.1	Solicitudes de registro, catálogo, perfil y compras.
/ws	Servidor de WebSockets (host interno:8765)	HTTP/1.1 + Upgrade	Conexión persistente para notificaciones en tiempo real (compras aprobadas/rechazadas, cambios de inventario).
/*	Frontend estático (equipo 1)	HTTP/1.1	Entrega de la interfaz web de la librería, si aplica servirla desde el mismo proxy.

Ejemplo de configuración (Caddyfile)

```
example.local {
  encode gzip

  @api path /api/*
  handle @api {
    reverse_proxy api-rest:3000
  }

  @ws path /ws*
  handle @ws {
    reverse_proxy websocket-server:8765
  }

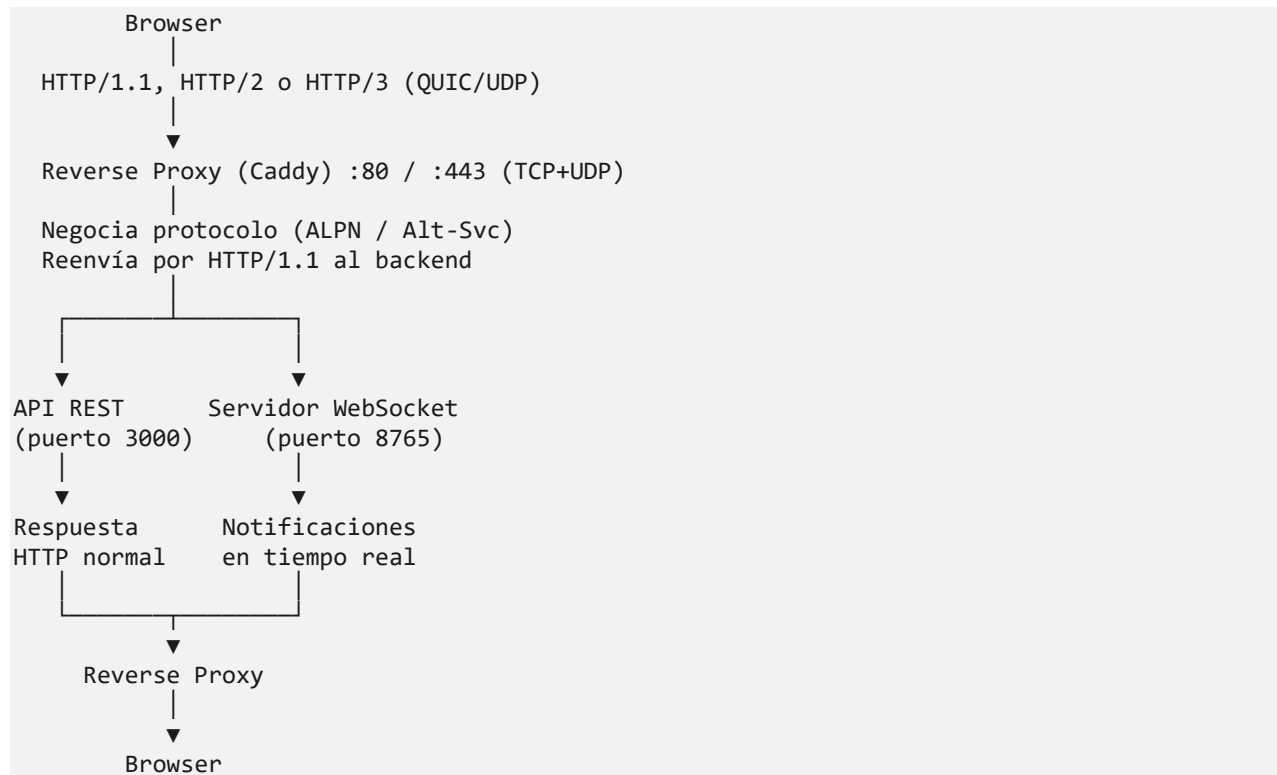
  handle {
    reverse_proxy frontend:8080
  }
}
```

Códigos HTTP relevantes

Código	Descripción
200 OK	La solicitud fue reenviada y respondida correctamente por el backend.
101 Switching Protocols	El proxy completó el handshake de actualización a WebSocket con el cliente.

Código	Descripción
502 Bad Gateway	El backend correspondiente (API REST o WebSocket) no respondió o rechazó la conexión.
504 Gateway Timeout	El backend tardó demasiado en responder dentro del tiempo configurado.

DIAGRAMA DEL FLUJO DE COMUNICACIÓN



CONCEPTOS A EXPLICAR EN LA DEMOSTRACIÓN

¿Qué es un Reverse Proxy?

Un Reverse Proxy es un servidor que se ubica delante de uno o más servidores backend y recibe las solicitudes de los clientes en su nombre, reenviándolas al servicio interno adecuado y devolviendo la respuesta al cliente como si proviniera del propio proxy. El cliente nunca se comunica directamente con el backend real.

Ventajas dentro de una arquitectura distribuida

- Punto único de entrada: simplifica la seguridad y el enrutamiento, ya que los servicios internos quedan ocultos de la red externa.
- Desacopla el protocolo del cliente del protocolo del backend: permite modernizar la capa de transporte (HTTP/3) sin modificar los servicios internos.
- Balanceo de carga y tolerancia a fallos, si se agregan múltiples instancias de un mismo backend.
- Terminación TLS centralizada: certificados y cifrado gestionados en un solo lugar.
- Facilita el versionado, la observabilidad y el registro (logging) centralizado de todas las solicitudes entrantes.

Diferencias entre HTTP/1.1, HTTP/2 y HTTP/3

Versión	Transporte	Características principales
HTTP/1.1	TCP	Texto plano, una solicitud a la vez por conexión (o pipelining limitado), sin multiplexación real; sufre bloqueo de línea (head-of-line blocking) a nivel de conexión.
HTTP/2	TCP	Binario, multiplexación de múltiples streams sobre una misma conexión TCP, compresión de encabezados (HPACK); aún expuesto al head-of-line blocking a nivel de TCP.
HTTP/3	QUIC sobre UDP	Mantiene la multiplexación de HTTP/2 pero sobre QUIC, eliminando el head-of-line blocking a nivel de transporte, con cifrado integrado y establecimiento de conexión más rápido (0-RTT).

Relación entre HTTP/3, QUIC y UDP

HTTP/3 es la versión de HTTP diseñada para funcionar sobre QUIC, un protocolo de transporte que a su vez se construye sobre UDP en lugar de TCP. QUIC incorpora en el espacio de usuario las funciones que TCP ofrecía a nivel de sistema (control de congestión, retransmisión, orden de paquetes) y agrega cifrado obligatorio (TLS 1.3 integrado) y multiplexación de streams independientes, de modo que la pérdida de un paquete de un stream no bloquea a los demás. Al usar UDP como base, QUIC evita las limitaciones estructurales de TCP y permite conexiones más rápidas de establecer y más resilientes ante pérdida de paquetes.

Cómo se mantiene la misma semántica HTTP entre versiones

Independientemente de si el transporte es HTTP/1.1, HTTP/2 o HTTP/3, la semántica de HTTP permanece idéntica: los mismos métodos (GET, POST, PUT, DELETE, etc.), los mismos encabezados (aunque en HTTP/2 y HTTP/3 se codifiquen de forma binaria y comprimida en lugar de texto plano) y los mismos códigos de estado (200, 404, 500, etc.). Lo que cambia entre versiones es únicamente la forma en que los mensajes se serializan y transportan por la red; la aplicación (en este caso, la API REST y el servidor de WebSockets) no necesita conocer ni adaptarse a la versión de HTTP utilizada por el navegador, ya que el Reverse Proxy se encarga de traducir el transporte y preservar la semántica en ambos extremos.

REPOSITORIO Y ENTREGABLES

- Repositorio independiente del módulo de Reverse Proxy, integrado posteriormente al repositorio principal mediante Git Submodules.
- Dockerfile con el servidor Caddy configurado y listo para ejecutarse.
- .env.example con las variables de configuración necesarias (hosts y puertos de los backends, dominio, etc.).
- README.md con instrucciones de ejecución y de integración con el resto de la plataforma.
- Incorporación final del servicio al docker-compose.yml del repositorio principal del proyecto.